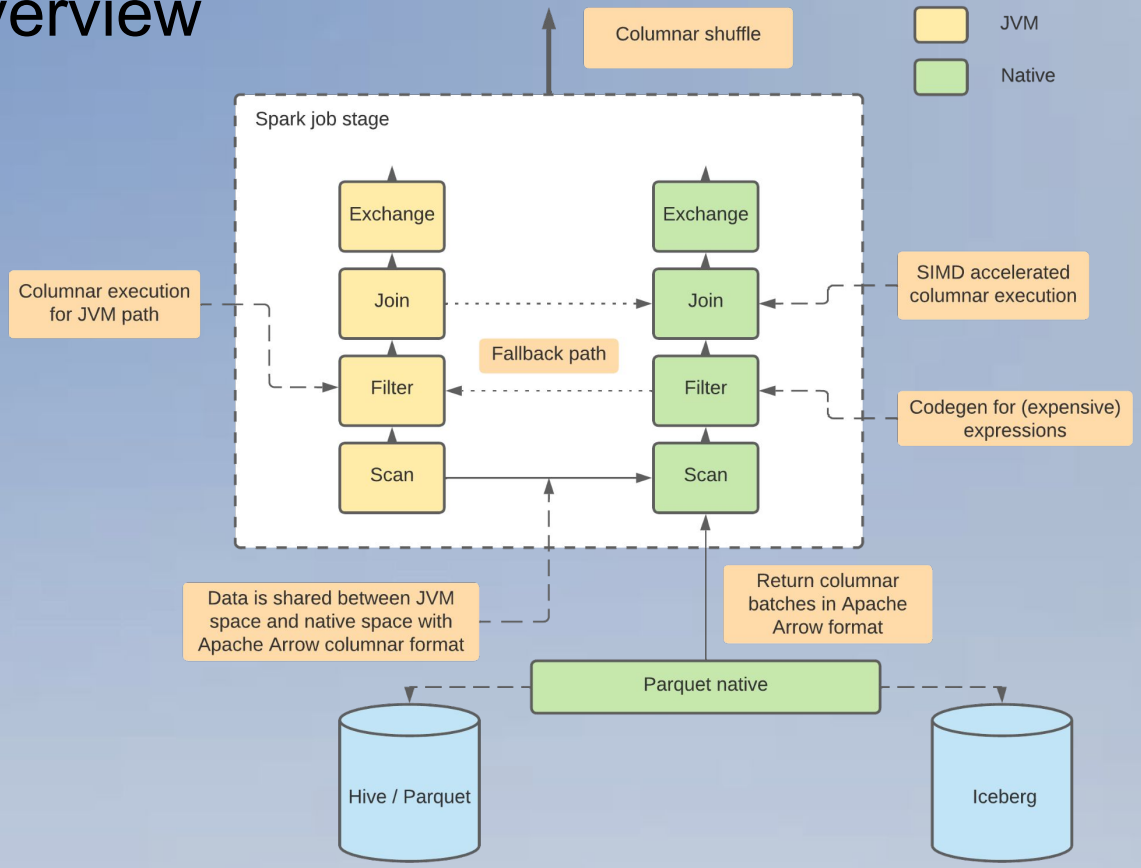
Accelerating Spark SQL with Apache Comet

Native Execution of Analytical Queries in Spark

EC528 Cloud Computing Milestone III

Members: Pranav Chintareddy, Adam Yanai, Yongyi Xie, Joseph Attie

Project Overview



https://datafusion.apache.org/comet/about/index.html

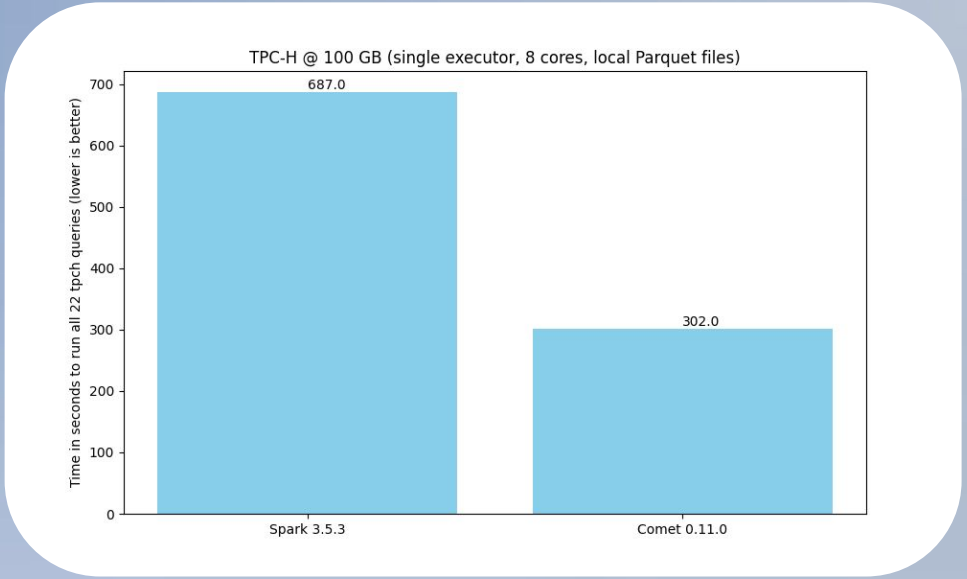
Milestone I Recap

Key Idea

- Execute natively in Rust, not JVM
- Reduce:
  - JVM overhead
  - Garbage collection
  - Latency

Results

  - ~70%+ operators executed natively (TPC-H)
  - Significant latency improvements



https://datafusion.apache.org/comet/contributor-guide/benchmark-results/tpc-h.html

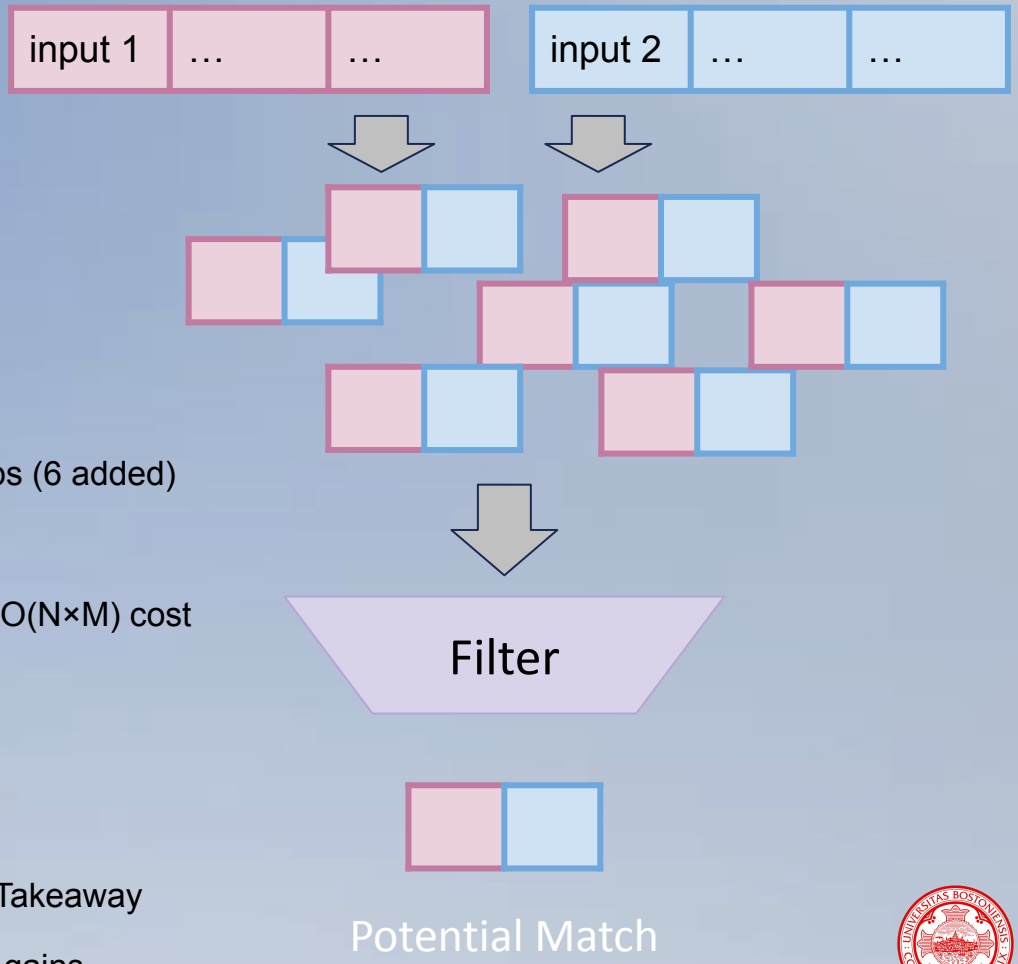
Milestone II Recap

Key Improvements

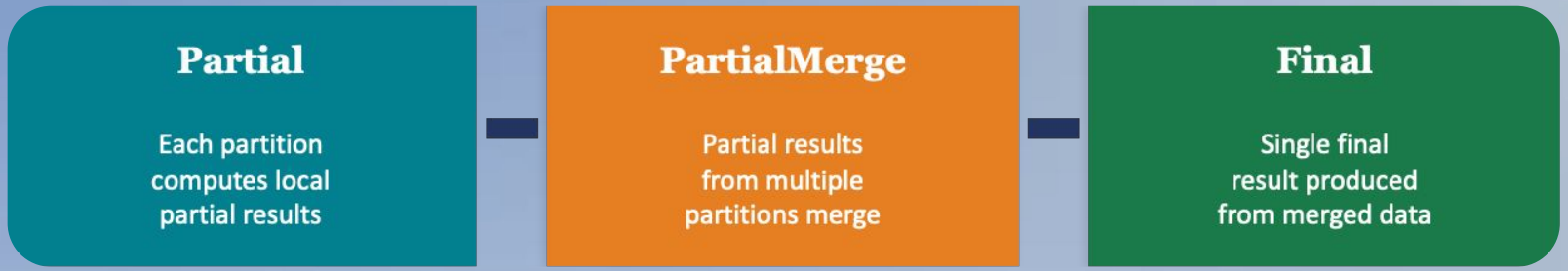
1. Anti Join Support (NOT IN / NOT EXISTS)
    - Implemented null-aware anti join logic in native
    - Enabled full native execution for Q16, Q22
2. New Native Operators
    - Aggregates (max_by, min_by) + Scalar math ops (6 added)
3. Join Optimization (Q21 bottleneck)
    - Problem: SMJ generates all candidate pairs → O(N×M) cost
    - Fix: apply filter during merge + early stop

Results

- Up to 10× speedup (Q21)
- Reduced memory + unnecessary computation Takeaway
- Minimizing fallback + optimizing joins = biggest gains



Fixing the PartialMerge Aggregation Fallback



Comet only knew about Partial and Final. When it saw PartialMerge it hit a catch-all case and returned None, handing the entire aggregation back to the JVM. This caused ALL of q16, q94, and q95 to partially fall back.

The fix required changes in 3 separate layers of the codebase: the protocol definition, the Scala translation layer, and the Rust execution engine. None of the 3 layers knew PartialMerge existed.

Layer 1: The Protobuf Definition

Protobuf is how Spark (JVM side) sends query plans to Comet's Rust engine. It's like a shared language between the two sides — if a concept isn't defined in the protobuf, neither side can talk about it.

PartialMerge was simply missing from the enum definition. It was never added when the aggregation mode support was first built.

Add PartialMerge to enum →

```
enum AggregateMode {
  Partial = 0;
  Final = 1;
  PartialMerge = 2;
}
```

Layer 2: The Scala Translation Layer

This is the Scala code that looks at a Spark query plan and decides whether each operator can be handed to Comet or not.

For aggregation, it reads the mode of each aggregate expression and maps it to a Comet-equivalent. PartialMerge hit the catch-all case and returned None — meaning 'fall back to Spark'.

Add PartialMerge:

```
val mode = modes.head match {
  case Partial      => CometAggregateMode.Partial
  case Final        => CometAggregateMode.Final
  case PartialMerge => CometAggregateMode.PartialMerge
  case _ => return None
}
```

Layer 3: The Rust Execution Planner

This is the Rust code that receives the serialized plan from Scala and builds the actual native execution plan using DataFusion.

The mode integer (0, 1, 2) comes in over the wire and gets mapped to DataFusion's AggregateMode enum. But DataFusion doesn't have PartialMerge — its closest equivalent is Final, which combines partial results.

So PartialMerge (mode == 2) maps to DFAggregateMode::Final in DataFusion.

```
let mode = if agg.mode == 0 {
    DFAggregateMode::Partial
} else {
    DFAggregateMode::Final
};
```

```
let mode = if agg.mode == 0 {
    DFAggregateMode::Partial
} else if agg.mode == 2 {
    DFAggregateMode::Final  // PartialMerge
} else {
    DFAggregateMode::Final
};
```

PartialMerge Queries - Current State

The fallback message: [COMET: Unsupported aggregation mode PartialMerge] ...no longer appears in any of the 3 queries. The fix successfully routes PartialMerge through Comet's native engine.

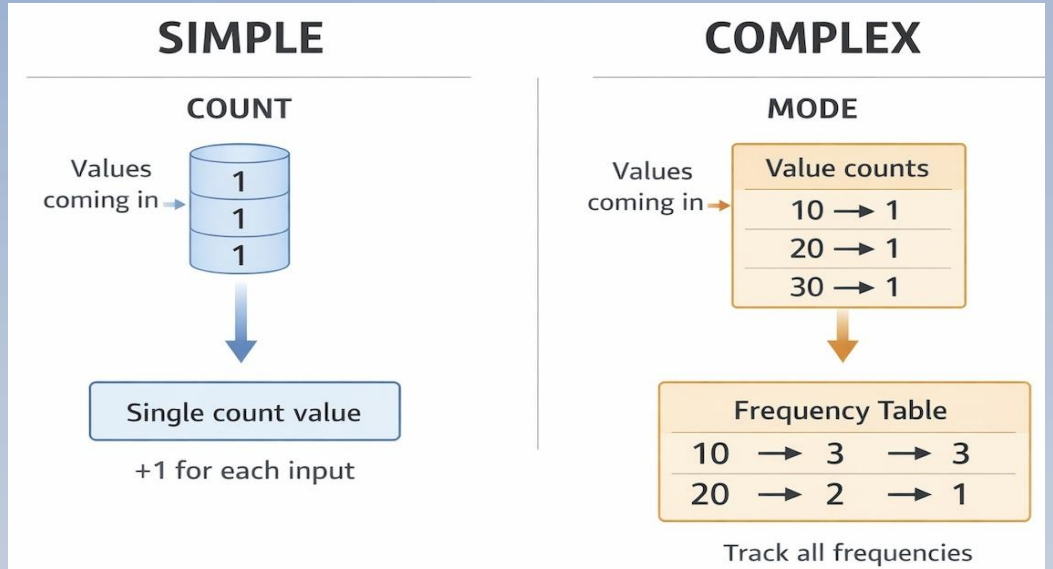
However, a deeper problem appeared: [COMET: cannot resolve cs_ext_ship_cost among cs_order_number, sum#897...]

Comet is now trying to execute the merge stage but can't find original column names, as they've already been compressed into partial buffers.

This is the next layer of the problem (column binding in merge stage). Fixing it requires teaching Comet's expression resolver to handle PartialMerge's different input schema.

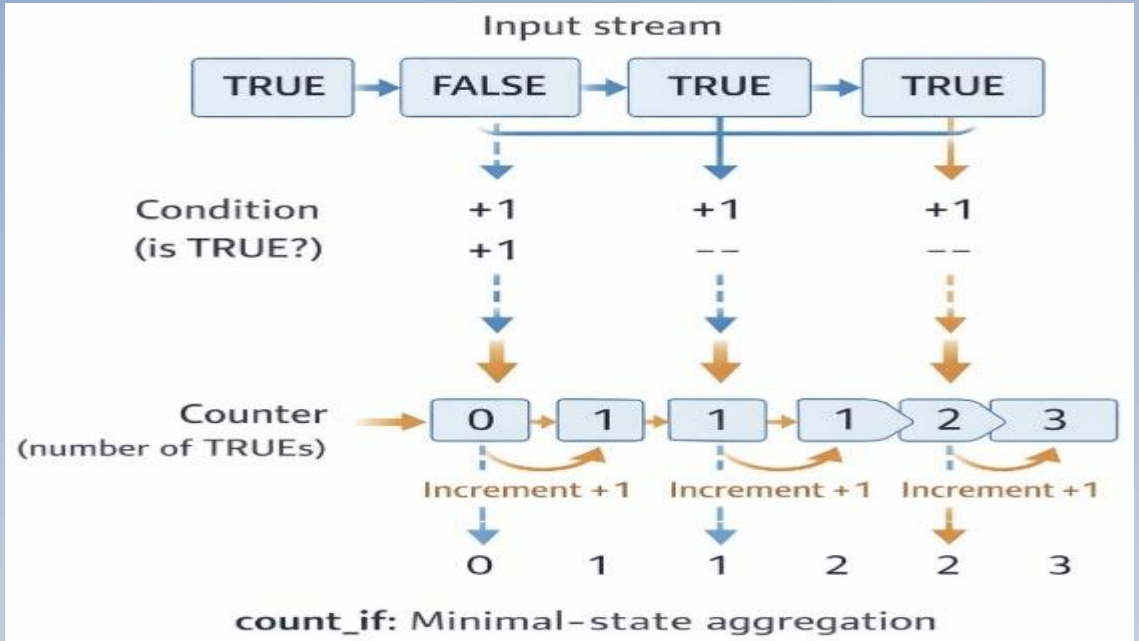
Not All Aggregations are Equal

- Some aggregations track a single running value
- Others must track a distribution of values to remember how often each value appears
- This creates a fundamental difference in how much information must be stored & how complex the computation becomes…



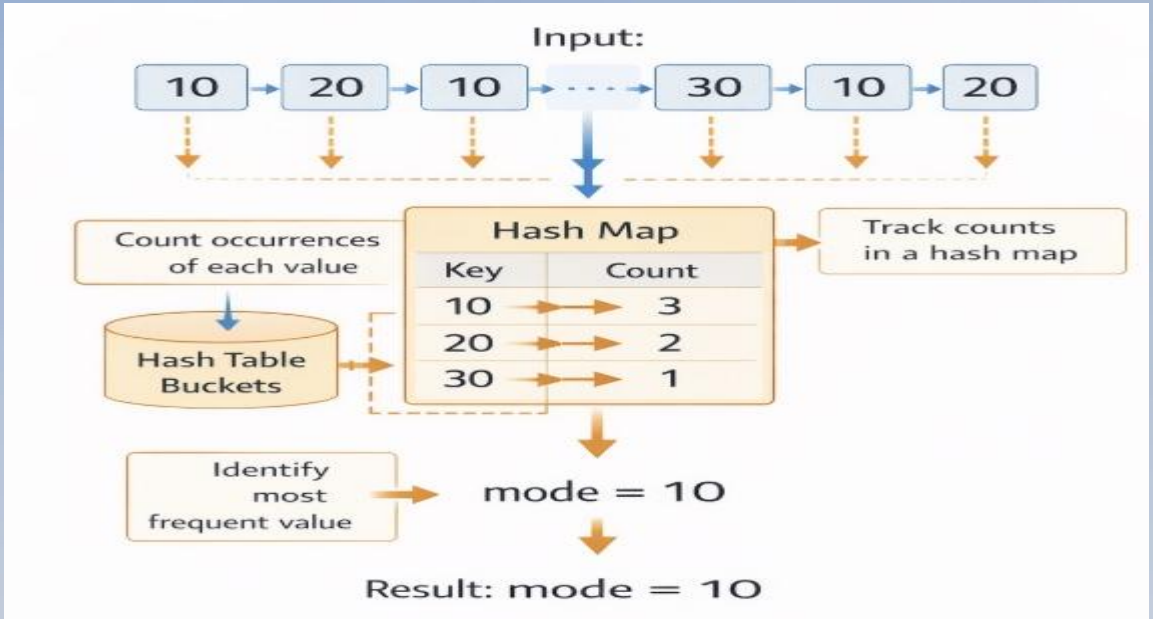
count_if(): Minimal-State Aggregation

- Uses a single counter → increments on valid condition
- No need to remember past values — only the total count
- Merge is trivial → add partial counts



mode(): Frequency-Based Aggregation

- Requires tracking how often each value appears
- Uses a frequency counter (value → count)
- Final result = value with maximum frequency



Fixing Inefficient Cached Table Reads

Problem: Currently, Cached tables require unnecessary format conversions due to extra Spark↔Comet transitions when Comet is enabled

This causes: extra overhead, inefficient execution, loss of control over how data is handled.

Key Insight:

The issue here is not caching itself

It is who controls the cache format and scan path for in-memory cache

Our Approach: Replace default behavior with a Comet-controlled cache pipeline & ensure cached data stays in a native-friendly format end-to-end.

What We Built: A Custom Cache Pipeline

1) Continuous Native Execution

Cached data is decoded directly into Arrow-based columnar batches usable by Comet

Avoids going through JVM's row-based scan and conversion steps

2) Precise Data Access

Only required columns are read at runtime

Avoids processing unused cached data

3) Direct Control Over Execution Flow

Comet now controls how cached data is: decoded, transformed and passed to downstream operators

Key Takeaways

Thank you!

Quiz Question:

What was the main reason queries fell back to Spark for PartialMerge aggregation?

A) PartialMerge was too slow in native execution

B) PartialMerge was missing from Comet’s implementation

C) Spark could not process PartialMerge at all

D) PartialMerge required more memory than available